Operating Systems Lab – GL-8

Threads & Multithreading

Lab Objectives

After completing this lab, students will be able to:

- create multiple threads in a program
- understand the concept of multithreading
- pass parameters to threads
- return values from threads
- cancel threads instantly or safely
- get and set thread attributes
- divide a large problem among multiple threads to improve performance

Introduction to Threads

A thread is a flow of execution inside a process.

A single process can contain multiple threads, where each thread performs a separate task independently.

A thread is also known as a: **Lightweight Process**

Real Life Understanding

Suppose we have an Online Food Delivery System.

When a customer places an order, the system performs multiple tasks at the same time:

- one task prepares food
- one task verifies payment
- one task assigns a rider
- one task sends notifications

If all tasks are executed one by one, the system becomes slow.

To improve speed and performance, the application divides the work among multiple threads.

This concept is called: **Multithreading**

Why Multithreading?

Multithreading is used to:

- improve performance
- execute multiple tasks simultaneously
- utilize CPU efficiently
- reduce execution time

Process vs Thread

Process	Thread
Heavyweight	Lightweight
Separate memory space	Shared memory space
Slow context switching	Faster context switching
Communication is difficult	Communication is easier

Shared Resources in Threads

Threads inside the same process share:

- code section
- data section
- files
- operating system resources

However, each thread has its own:

- stack
- registers
- program counter (PC)

Advantages of Threads

- Responsiveness
- Faster context switching
- Efficient CPU utilization
- Resource sharing
- Easy communication between threads
- Increased throughput
- Reduced overhead compared to processes

- Better performance in multi-core systems
- Parallel execution of tasks
- Improved application efficiency

Pthread Library

Linux uses the:

pthread

library for thread programming.

Required Header File

```
#include <pthread.h>
```

Compilation Command

```
gcc file.c -pthread
```

The `-pthread` flag links the pthread library with the program.

pthread_create()

The `pthread_create()` function is used to create a new thread.

Syntax

```
pthread_create(&thread_id, attr, function_name, argument);
```

Parameters Explanation

Parameter	Description
thread_id	stores thread ID
att	thread attributes
function_name	function executed by thread
argument	value passed to thread

How pthread_create() Works

When `pthread_create()` is called:

- a new thread is created
- the thread starts execution independently
- the given function starts running

pthread_join()

The `pthread_join()` function waits for a thread to complete execution.

Syntax

```
pthread_join(thread_id, &retval);
```

Purpose of pthread_join()

- waits for thread completion
- receives returned value from thread
- synchronizes threads with the main program

Passing Parameters to Threads

Threads can receive:

- integers
- arrays
- strings
- structures

Parameters are generally passed using:

```
void *
```

Returning Values from Threads

Threads can return values using:

```
pthread_exit()
```

Returned values are collected using:

```
pthread_join()
```

Thread Cancellation

Sometimes a running thread must be terminated before completion.

For this purpose:

```
pthread_cancel()
```

is used.

Syntax

```
pthread_cancel(thread_id);
```

Cancellation State

A thread can either:

- allow cancellation
- block cancellation

Enable Cancellation

```
PTHREAD_CANCEL_ENABLE
```

The thread can be cancelled.

Disable Cancellation

`PTHREAD_CANCEL_DISABLE`

The thread cannot be cancelled until cancellation is enabled again.

Cancellation Types

There are two types of thread cancellation.

1. Asynchronous Cancellation

In asynchronous cancellation, the thread terminates immediately after receiving the cancellation request.

`PTHREAD_CANCEL_ASYNCHRONOUS`

2. Deferred Cancellation

In deferred cancellation, the thread terminates only at a safe cancellation point.

`PTHREAD_CANCEL_DEFERRED`

Safe cancellation point:

```
pthread_testcancel();
```

Functions Used in Thread Cancellation

Function	Purpose
`pthread_cancel()`	cancel thread
`pthread_setcancelstate()`	enable/disable cancellation
`pthread_setcanceltype()`	set cancellation type
`pthread_testcancel()`	define cancellation point

Thread Attributes

Thread attributes are managed using:

`pthread_attr_t`

Thread attributes control thread behavior during thread creation.

Important Attribute Functions

Function	Purpose
pthread_attr_init()	initialize attributes
pthread_attr_destroy()	destroy attributes
pthread_attr_getstacksize()	get stack size
pthread_attr_setstacksize()	set stack size

Stack Size in Threads

Each thread has its own stack memory.

We can:

- check stack size
- modify stack size before thread creation

Practical Example 1

Thread Creation & Termination

Scenario

Suppose a company server handles multiple client requests simultaneously.

Each request is handled by a separate thread.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 5

void* clientRequest(void* arg)
{
    long id = (long)arg;

    printf("Client Thread %ld is executing\n", id);
```

```
    pthread_exit(NULL);
}

int main()
{
    pthread_t threads[NUM_THREADS];

    for(long i = 0; i < NUM_THREADS; i++)
    {
        printf("Creating Thread %ld\n", i);

        pthread_create(&threads[i], NULL,clientRequest, (void*)i
        );
    }

    for(int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Practical Example 2

Passing Parameters & Returning Values

Scenario

Suppose a banking system divides transactions among multiple threads.

Each thread calculates partial sums and returns the result to the main thread.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

void* add(void* ptr)
{
    int* numbers = (int*)ptr;

    int* result = malloc(sizeof(int));

    *result = numbers[0] + numbers[1];

    pthread_exit(result);
}

int main()
{
```

```
    pthread_t t1, t2, t3;

    void *ret1, *ret2, *ret3;

    int values[] = {10,20,30,5,15,30};

    pthread_create(&t1, NULL,add, &values[0]);

    pthread_create(&t2, NULL,add, &values[2]);

    pthread_create(&t3, NULL,add, &values[4]);

    pthread_join(t1, &ret1);
    pthread_join(t2, &ret2);
    pthread_join(t3, &ret3);

    int total =
        *((int*)ret1) +
        *((int*)ret2) +
        *((int*)ret3);

    printf("Total Sum = %d\n", total);

    free(ret1);
    free(ret2);
    free(ret3);

    return 0;
}
```

Practical Example 3

Thread Cancellation

Scenario

Suppose a download manager is downloading files.

One thread uses:

- asynchronous cancellation

Another thread uses:

- deferred cancellation

This helps understand the difference between immediate and safe thread cancellation.

Code

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

void* asyncThread()
{
    pthread_setcanceltype(
        PTHREAD_CANCEL_ASYNCHRONOUS,
        NULL
    );

    while(1)
    {
        printf("Asynchronous Thread Running\n");
    }
}

void* deferredThread()
{
    pthread_setcancelstate(
        PTHREAD_CANCEL_ENABLE,
        NULL
    );

    pthread_setcanceltype(
        PTHREAD_CANCEL_DEFERRED,
        NULL
    );

    while(1)
    {
        printf("Deferred Thread Running\n");

        pthread_testcancel();
    }
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, asyncThread, NULL);

    pthread_create(&t2, NULL, deferredThread, NULL);

    sleep(5);

    pthread_cancel(t1);

    sleep(5);

    pthread_cancel(t2);
```

```
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```

Practical Example 4

Thread Attributes & Stack Size

Scenario

Suppose a system administrator wants to:

- check thread stack size
- modify stack size before thread creation

Code

```
#include <stdio.h>
#include <pthread.h>

void* task(void* arg)
{
    printf("Thread Executing\n");

    pthread_exit(NULL);
}

int main()
{
    pthread_t t1;

    pthread_attr_t attr;

    size_t stacksize;

    pthread_attr_init(&attr);

    pthread_attr_getstacksize(&attr,&stacksize
    );

    printf("Current Stack Size: %ld\n",
            stacksize);

    pthread_attr_setstacksize(  &attr,1024 * 1024 * 16);

    pthread_create(&t1,&attr,task,NULL
    );

    pthread_join(t1, NULL);
```

```
    pthread_attr_destroy(&attr);

    return 0;
}
```

Graded Practice Task

Task 1 – Smart University Result System (Multithreading + Stats)

A university system processes student marks for a class. You are given a list of marks as command-line input.

Scenario:

The system must process the data in parallel using threads:

- One thread calculates **average marks**
- One thread finds **highest marks**
- One thread finds **lowest marks**

Input Example:

```
./a.out 78 85 90 66 72 88 95
```

Output Example:

```
Average Marks: 82
Highest Marks: 95
Lowest Marks: 66
```

Requirements:

- Use multiple threads
- Each thread performs only one task
- Main thread waits for all results

Task 2 – Multi-Threaded Online Exam System (Cancellation Concept)

Scenario:

An online exam system runs multiple background monitoring threads:

- One thread monitors cheating detection
- One thread monitors network status
- One thread monitors time remaining

If the exam ends early, all monitoring threads must be stopped.

Requirements:

- Create 3 threads
- Use thread cancellation (`pthread_cancel`)
- One thread should stop immediately (asynchronous behavior)
- One thread should stop at safe point (`pthread_testcancel`)
- Demonstrate clean thread termination

Expected Output:

```
Monitoring cheating...
Monitoring network...
Monitoring time...

Exam ended → stopping all threads...
```

Task 3 – File Search Engine (Multithreading + Early Exit)

Scenario:

A file search system is designed to search a keyword inside a large file using multiple threads.

- File is divided into parts
- Each thread searches in its own part
- If any thread finds the keyword, all other threads must stop immediately

Input Example:

```
File: data.txt
Search String: "Operating"
Threads: 4
```

Requirements:

- Use multiple threads
- Pass file segments to threads
- If string is found:
    - print line number + column number
    - cancel all other threads immediately
- Use `pthread_cancel()` and cancellation handling

Expected Output:

```
String found at Line 12, Column 5
Stopping all threads...
```